

Caso de estudio #1

"Smart Assist"

Oscar Chavarría Hernández

Universidad CENFOTEC

Gestión de Riesgos

Profesor: JORGE ROBERTO MORA
SEGURA

2025

Gestión de riesgos: Proyecto "Smart Assist"

Portada	1
Indice	2
Informe de Gestión de Riesgos	3
Identificación de riesgos:	3
Evaluación de riesgos.	3
Propuesta de mitigación.	3
Revisión Continua:	4
Reflexión final	4

Introducción

El desarrollo del proyecto "Smart Assist", una aplicación avanzada que integra inteligencia artificial y servicios en la nube representa un reto significativo tanto en términos tecnológicos como de gestión. Dado su alto nivel de complejidad e innovación, es fundamental anticipar y gestionar proactivamente los riesgos que podrían comprometer la seguridad, calidad, continuidad operativa o el cumplimiento de los plazos establecidos.

Con este objetivo, se ha realizado un análisis exhaustivo del contexto del proyecto, identificando diez riesgos clave que abarcan desde fallas técnicas (como problemas de integración de modelos de IA o latencias en la nube), hasta riesgos organizacionales (como la rotación de personal o la comunicación en equipos remotos). Estos riesgos han sido evaluados en función de su probabilidad de ocurrencia y su impacto potencial, permitiendo clasificarlos en niveles de criticidad para enfocar los esfuerzos de mitigación.

En este informe se presenta no solo la identificación y evaluación cualitativa de los riesgos, sino también una propuesta detallada de mitigación, enfocada especialmente en los dos riesgos considerados críticos: *fallas en la integración de inteligencia artificial* y *brechas de seguridad en la nube (AWS/S3)*. Además, se describen los mecanismos de monitoreo y revisión continua que se implementarán durante los próximos seis meses de desarrollo, con el fin de adaptar las estrategias de gestión de riesgos a los cambios del entorno y al progreso del proyecto.

Este enfoque integral busca garantizar la resiliencia del sistema, proteger los activos digitales del proyecto, y asegurar una experiencia confiable para los usuarios finales desde el primer día de lanzamiento.

Informe de Gestión de Riesgos
Proyecto: Smart Assist
Responsable: Gestor de Riesgos
Empresa: Cenfo TechInnovators
Fecha: Junio 2025

1. Identificación de Riesgos

Tras el análisis del contexto del proyecto "Smart Assist", se han identificado los siguientes 10 riesgos potenciales que podrían afectar la seguridad, calidad, continuidad o cumplimiento de los plazos de desarrollo.

Nº	Riesgo	Descripción
1	Fallas en la integración de IA	Los modelos de inteligencia artificial pueden no adaptarse correctamente a escenarios reales.
2	Brechas de seguridad en la nube (AWS/S3)	Riesgo de exposición de datos sensibles debido a malas configuraciones o ciberataques.
3	Demora en pruebas de QA	Posibles retrasos en la detección de errores críticos que afecten la estabilidad del producto.
4	Problemas de compatibilidad entre iOS y Android	Riesgo de funcionalidad desigual o errores en una de las plataformas.
5	Rotación de personal clave	Pérdida de conocimiento del sistema si se marcha personal con funciones críticas.
6	Cambios en los requisitos del cliente	Solicitudes fuera del alcance inicial que puedan desestabilizar el cronograma.
7	Sobrecarga de servidores tras el lanzamiento	La arquitectura podría no soportar una carga de usuarios elevada.
8	Latencia en servicios en la nube	Degradación del rendimiento por retrasos en el procesamiento remoto.
9	Comunicación ineficiente en el equipo remoto	Malentendidos o descoordinación que afecten tiempos y calidad.
10	Fallas en la publicación en App Store / Google Play	Riesgo de rechazo de la app por incumplimiento de normativas.

2. Evaluación de Riesgos

A continuación, se presenta la evaluación cualitativa de los riesgos identificados, considerando la probabilidad de ocurrencia y su impacto en el proyecto.

Nº	Riesgo	Probabilidad	Impacto	Clasificación
1	Fallas en la integración de IA	Alta	Alta	Crítico
2	Brechas de seguridad en la nube	Alta	Alta	Crítico
3	Demora en pruebas de QA	Alta	Media	Alto
4	Compatibilidad iOS/Android	Media	Alta	Alto
5	Rotación de personal clave	Media	Alta	Alto
6	Cambios en requisitos del cliente	Media	Alta	Alto
7	Sobrecarga del backend	Media	Alta	Alto
8	Latencia en servicios en la nube	Media	Media	Medio
9	Comunicación ineficiente (equipo remoto)	Media	Media	Medio
10	Rechazo en tiendas de apps	Baja	Alta	Medio

3. Propuesta de Mitigación

Se detallan a continuación los planes de mitigación para los riesgos clasificados como críticos:

Riesgo 1: Fallas en la integración de IA

- **Medidas preventivas:**

- Validación continua de modelos en entornos de prueba que simulen condiciones reales de uso.
- Aplicación de pruebas unitarias, de regresión y funcionales orientadas específicamente a los módulos de IA.
- Entrenamiento de modelos con datos representativos y actualizados.

- **Acciones correctivas:**

- Sustitución o reentrenamiento de modelos que presenten errores.
- Desactivación temporal de funcionalidades basadas en IA para evitar impactos mayores en la experiencia del usuario.

- **Planes de contingencia:**

1. Incorporación de funciones manuales o versiones básicas sin IA

- **Cómo:** Diseñar desde el inicio una estructura modular que permita aislar o desactivar los componentes de IA sin afectar las funcionalidades básicas de la aplicación. Esto implica tener una interfaz de usuario alternativa que no dependa de predicciones o respuestas generadas automáticamente, permitiendo al usuario operar la aplicación de forma manual.
- **Por qué:** Si la IA falla o da resultados incoherentes, es fundamental que el usuario pueda seguir utilizando la app para tareas básicas (como enviar comandos simples, acceder a configuraciones, o recibir ayuda técnica), lo cual mantiene la percepción de continuidad del servicio y evita la pérdida de usuarios.

2. Despliegue de actualizaciones rápidas mediante pipelines de integración continua (CI/CD)

- **Cómo:** Configurar un sistema CI/CD (por ejemplo, GitHub Actions, GitLab CI o AWS CodePipeline) que permita validar, compilar, testear y desplegar actualizaciones del modelo o sus configuraciones en cuestión de minutos, sin intervención manual prolongada.
- **Por qué:** En escenarios donde la IA causa errores o mal funcionamiento, es crítico poder responder con rapidez para evitar daños a la experiencia del usuario o reputación del producto. Un pipeline CI/CD automatizado reduce significativamente el tiempo entre detección y corrección.

3. Versionado y rollback automático de modelos de IA

- **Cómo:** Almacenar todas las versiones entrenadas de los modelos en un repositorio de modelos (por ejemplo, MLflow, TensorFlow Model Registry o S3 con versionado activado), permitiendo hacer rollback automático a una versión anterior estable mediante una simple configuración.
- **Por qué:** Si una nueva versión del modelo causa errores en producción, se puede revertir de inmediato sin necesidad de reentrenar o recodificar. Esto minimiza el tiempo de inactividad y mantiene la estabilidad operativa.

4. Activación de monitoreo con umbrales y alertas para detección temprana de fallos

- **Cómo:** Implementar monitoreo en tiempo real sobre el rendimiento de los modelos (precisión, errores, tiempos de respuesta, etc.) con herramientas como Prometheus + Grafana o AWS CloudWatch, configurando umbrales que, si se superan, disparen alertas o desactiven la IA automáticamente.
- **Por qué:** Detectar patrones anómalos o comportamientos erráticos permite anticiparse al fallo total del sistema. Un sistema de alertas permite iniciar el plan de contingencia antes de que los usuarios lo noten.

5. Integración de IA de terceros como respaldo temporal

- **Cómo:** Utilizar APIs de proveedores externos como Google Cloud AI, Microsoft Azure Cognitive Services o OpenAI para ciertas funciones críticas (reconocimiento de voz, procesamiento de lenguaje natural) en caso de que el modelo propio falle.
- **Por qué:** Esto permite mantener parte de la funcionalidad con calidad aceptable mientras se solucionan los problemas internos, evitando que la aplicación quede completamente inutilizada.

6. Modo “Degradado” o “Limitado”

- **Cómo:** Activar un estado de la aplicación que limite las funcionalidades que dependan directamente de la IA y mantenga activas solo aquellas que son estables o sin IA, mostrando un mensaje al usuario informando que la app está en “modo limitado”.
- **Por qué:** La transparencia mejora la confianza del usuario y, al mismo tiempo, se protege la estabilidad del sistema y se evita una mala experiencia causada por respuestas erráticas de la IA.

Riesgo 2: Brechas de seguridad en la nube (AWS/S3)

- **Medidas preventivas:**
 - Configuración adecuada de roles, permisos y políticas de acceso (IAM) en los servicios de AWS.
 - Implementación de cifrado de datos en tránsito y en reposo.
 - Revisión periódica de logs y auditorías automáticas de configuración.
- **Acciones correctivas:**
 - Aislamiento inmediato del entorno afectado.
 - Activación del protocolo de respuesta ante incidentes (IRP).
 - Revocación de claves comprometidas y modificación de accesos.
- **Planes de contingencia:**

1. Backups automáticos y almacenamiento en múltiples regiones

Cómo: Se configuran copias de seguridad automáticas (backups) de los datos utilizando servicios como AWS Backup o funciones automáticas con AWS Lambda (una herramienta que permite ejecutar código sin necesidad de servidores). Estas copias se almacenan en diferentes ubicaciones geográficas (regiones de AWS).

Por qué: Si ocurre una brecha o falla en una región (por ejemplo, un ataque o desastre natural), los datos pueden recuperarse desde otra región sin interrupciones. Esto mejora la disponibilidad y la resiliencia del sistema.

2. Versionado habilitado en S3 (Object Versioning)

Cómo: El versionado es una función de Amazon S3 que guarda todas las versiones de un archivo cuando este se modifica o elimina.

Por qué: Si un archivo es sobrescrito por error o borrado durante una brecha de seguridad, se puede recuperar una versión anterior. Esto protege contra la pérdida o alteración de información.

3. Replicación entre buckets con políticas de acceso independientes

Cómo: Se configura la **replicación entre buckets**, que son contenedores de almacenamiento en la nube (S3). Esto significa que los archivos se copian automáticamente desde un bucket original a otro, en una región distinta. Las **políticas de acceso (IAM)** del bucket destino son diferentes y más restrictivas.

Por qué: Si el bucket principal es comprometido, el bucket replicado mantiene una copia segura con acceso limitado, lo que facilita la recuperación.

4. Infraestructura como código para restauración rápida

Cómo: Se utilizan herramientas como AWS CloudFormation o Terraform, que permiten definir toda la infraestructura (servidores, redes, permisos, etc.) en archivos de texto. Esto se conoce como **Infraestructura como Código (IaC)**.

Por qué: En caso de una brecha que destruya o comprometa la infraestructura, se puede reconstruir rápidamente desde estos archivos, sin errores humanos y con configuración estandarizada.

5. Simulacros regulares de recuperación y respuesta a incidentes

Cómo: Se realizan ejercicios internos donde se simula una brecha de seguridad. El equipo sigue los pasos del plan de contingencia para recuperarse, responder y restaurar los servicios. Esto incluye comunicación, análisis y validación.

Por qué: Practicar con anticipación permite detectar fallos en los planes, capacitar al personal y asegurar que todos sepan cómo actuar en una situación real.

6. Bucket “air-gapped” de solo escritura para datos críticos

Cómo: Se crea un bucket especial en S3 con acceso extremadamente limitado: solo se permite **escribir** datos, y no se permite leer, borrar o modificar nada, ni siquiera por administradores. Esto se logra con políticas de acceso (IAM) estrictas.

Por qué: Este bucket actúa como una bóveda inalterable que conserva información crítica. Incluso si todo lo demás se ve comprometido, este almacenamiento sigue siendo confiable.

4. Revisión Continua

Para asegurar la supervisión efectiva de los riesgos durante el resto del desarrollo del proyecto (seis meses hasta el lanzamiento), se establecen los siguientes mecanismos y herramientas:

Mecanismos de monitoreo

- **Reuniones quincenales de evaluación de riesgos** durante las revisiones de sprint, con análisis de desviaciones y actualización del registro de riesgos.

- **Registro dinámico de riesgos** centralizado (por ejemplo, en Confluence o JIRA), accesible a todos los responsables de área.
- **Indicadores clave de riesgo (KRIs)** definidos por tipo de riesgo (ej. tiempos de respuesta, incidencias de seguridad, rotación de personal, etc.).

Herramientas de soporte

- **JIRA:** Seguimiento de tareas, incidencias y riesgos identificados.
- **Confluence:** Repositorio de documentación, procedimientos y controles.
- **Slack (con bots):** Alertas automáticas sobre cambios críticos o incidentes en progreso.
- **AWS CloudWatch:** Monitoreo en tiempo real de rendimiento y disponibilidad de los servicios en la nube.
- **SonarQube:** Evaluación automatizada de la calidad del código fuente y detección de vulnerabilidades.

Recomendaciones Finales

Se recomienda priorizar la mitigación y supervisión activa de los riesgos críticos (falla en integración de IA y seguridad en la nube), además de establecer pruebas técnicas cruzadas para garantizar la compatibilidad y escalabilidad del producto. La gestión de riesgos deberá mantenerse como un proceso iterativo durante todo el ciclo restante del proyecto.

Reflexión final

Reflexione y responda las siguientes preguntas.

1- ¿Cuáles son los principales riesgos asociados con el desarrollo remoto de software y cómo pueden mitigarse?

Los principales riesgos incluyen la comunicación ineficiente, la falta de sincronización entre equipos, la pérdida de productividad, y problemas en la gestión de tareas y seguimiento del trabajo. Además, puede haber desafíos culturales y de zona horaria que afecten la colaboración.

Medidas de mitigación:

- Establecer canales de comunicación claros y regulares (como reuniones diarias, herramientas de mensajería y videoconferencias).
- Utilizar herramientas de gestión de proyectos como JIRA, Trello o Asana para asignar y seguir tareas.
- Establecer horarios de trabajo solapados para facilitar la colaboración entre zonas horarias distintas.
- Fomentar una cultura de documentación clara y actualizada para evitar malentendidos.
- Implementar revisiones de sprint frecuentes y retrospectivas para ajustar procesos.

2- ¿Qué riesgos específicos se presentan al trabajar con inteligencia artificial en aplicaciones móviles y cómo pueden gestionarse eficazmente?

Los riesgos más comunes incluyen la inexactitud de los modelos de IA, falta de adaptabilidad a entornos reales, consumo elevado de recursos del dispositivo móvil, y sesgos en los datos que afectan la equidad del sistema.

Gestión efectiva:

- Validar los modelos en escenarios reales antes del despliegue.
- Entrenar modelos con datos representativos y actualizados.

- Monitorear continuamente el rendimiento del modelo con métricas clave.
- Diseñar la aplicación con arquitectura modular para permitir desactivar funciones de IA sin afectar el resto de la app.
- Implementar mecanismos de rollback y versionado de modelos para recuperación rápida ante fallos.

3- ¿Cómo puede la seguridad en la nube, especialmente al utilizar servicios de AWS, afectar el lanzamiento de "Smart Assist"? ¿Qué medidas de control se pueden implementar?

La seguridad en la nube es crítica, ya que una brecha podría comprometer datos sensibles de usuarios o la funcionalidad de la aplicación. Un incidente de este tipo puede retrasar el lanzamiento, dañar la reputación y causar pérdida de usuarios.

Medidas de control:

- Configurar correctamente los roles y permisos (IAM) para controlar el acceso a los recursos.
- Usar cifrado en tránsito y en reposo para proteger los datos.
- Activar el versionado en buckets de S3 para recuperar archivos en caso de corrupción o eliminación.
- Implementar backups automáticos en múltiples regiones para garantizar la disponibilidad de datos.
- Realizar auditorías periódicas de seguridad y simulacros de respuesta ante incidentes.
- Usar infraestructura como código (IaC) para facilitar la recuperación rápida.

4- ¿Qué impacto tendría un retraso en el lanzamiento de "Smart Assist" en la feria tecnológica? ¿Cómo puede gestionarse este riesgo?

Un retraso podría significar la pérdida de una oportunidad clave para demostrar el producto a potenciales inversores, clientes o socios estratégicos. También puede afectar la percepción de confiabilidad de la empresa y su capacidad de cumplimiento.

Gestión del riesgo:

- Establecer un cronograma con márgenes de contingencia para etapas críticas del proyecto.
- Priorizar el desarrollo del núcleo funcional del producto para asegurar que una versión funcional mínima esté lista para la feria.
- Preparar una demo pregrabada o simulación interactiva que pueda presentarse si la app completa no está lista.
- Monitorear continuamente el progreso y aplicar metodologías ágiles para ajustar el alcance en función del tiempo disponible.

5- ¿Qué herramientas o prácticas de gestión de riesgos son más eficaces para un equipo de desarrollo distribuido en múltiples ubicaciones?

Herramientas eficaces:

- JIRA o Azure DevOps: para gestión de tareas, incidencias y seguimiento de riesgos.
- Confluence o Notion: como repositorio centralizado de documentación de riesgos y controles.
- Slack o Microsoft Teams: para comunicación rápida y notificaciones automáticas sobre incidentes.
- AWS CloudWatch o Datadog: para monitoreo en tiempo real de sistemas y generación de alertas.

Prácticas recomendadas:

- Revisión quincenal de riesgos durante las reuniones de sprint.
- Registro dinámico de riesgos accesible a todo el equipo.
- Definición de indicadores clave de riesgo (KRIs) para cada área.
- Simulacros regulares y sesiones de formación en gestión de riesgos.

Conclusión

La gestión de riesgos en proyectos de desarrollo de software, especialmente aquellos que integran inteligencia artificial y servicios en la nube como "Smart Assist", es un proceso esencial para garantizar el éxito técnico, operativo y comercial del producto. En entornos distribuidos y remotos, los riesgos se amplifican debido a la complejidad de la comunicación, la coordinación de equipos y la dependencia de infraestructuras externas como AWS.

La identificación proactiva de amenazas como fallas en la integración de IA, brechas de seguridad en la nube o retrasos en los plazos permite diseñar estrategias específicas de mitigación que aseguren la resiliencia del sistema. A través del uso de herramientas modernas de monitoreo, automatización, control de versiones y comunicación colaborativa, es posible mantener la estabilidad del proyecto incluso ante escenarios críticos.

Una planificación efectiva, combinada con prácticas ágiles y mecanismos de revisión continua, no solo permite reducir la probabilidad e impacto de los riesgos, sino que también fortalece la capacidad del equipo para adaptarse rápidamente a los cambios y entregar una solución robusta, segura y confiable desde el primer día. Esta disciplina se convierte, por tanto, en un pilar clave para el éxito del lanzamiento de "Smart Assist" y su posicionamiento en eventos estratégicos como ferias tecnológicas.